

Московский физико-технический институт
Высшая школа программной инженерии

Формальные языки и сложность вычислений
3 семестр, 2026/2027 уч. год

Лекция 1.
Сложность вычислений. Виды
вычислительных задач и ресурсов.

Лектор: Зухба Анастасия Викторовна
Автор: [Бессовестный копипастер](#)

1 Вычислительные задачи

Сейчас мы приведём примеры задач, которые нам бы очень хотелось вычислять на машинах Тьюринга. (Причём в идеале за время, полиномиально зависящее от размера входных данных).

- **Задача распознавания языка:** по слову x нужно выяснить, принадлежит ли оно языку $L \subset \Sigma^*$. Алгоритм должен возвращать 1, если $x \in L$, и 0, если $x \notin L$.
- **Задача вычисления функции:** по слову x нужно найти $f(x)$, где $f : \Sigma^* \rightarrow \Sigma^*$. То есть, для любого x алгоритм должен возвращать $f(x)$. Как правило мы считаем, что f всюду определена.
- **Задача поиска:** по слову x нужно найти слово y , такое что xRy , где $R \subset \Sigma^* \times \Sigma^*$ — некоторое отношение. То есть, на любом x алгоритм должен либо вернуть y , такой что xRy , либо сообщить, что такого y нет.

2 Вычислительные ресурсы

Ещё мы хотим оценивать, сколько ресурсов будет затрачено на ту или иную задачу. Рассматривать будем следующие ресурсы: время, память и длину программы.

- **Время работы:** поскольку мы изучаем алгоритмы, выполняемые пошагово, то нам будет удобно оценивать алгоритмы в этих самых шагах.
- **Память:** алгоритму необходимо пространство для промежуточных вычислений. Обычно в это пространство не включают вход программы.
- **Длина программы:** бывают ситуации, когда программа с более длинным текстом будет работать быстрее. Изучению этого фактора посвящена теория колмогоровской сложности.

Минимизировать затрачиваемые ресурсы нам могут помочь следующие штучки:

- **Подсказка:** если машина получает небольшую подсказку, зависящую только от длины входа, она может существенно ускориться. Более того, с подсказкой можно решить даже некоторые невычислимые функции. Использование подсказки моделирует ситуацию, в которой длина входа заранее известна.
- **Случайность:** некоторые задачи при современном уровне знаний решаются гораздо быстрее, если применить вероятностный алгоритм вместо детерминированного. Как правило, расплатой за это ускорение будет маленькая, но положительная вероятность ошибки.
- **Интерактивность:** часто проверка результата может быть значительно проще его получения, поэтому можно использовать делегирование вычислений.
- **Специальные модели:** недетерминированные, альтернирующие и квантовые вычисления.
- **Оракул:** вычисления с оракулом позволяют про любое слово за один шаг узнавать, принадлежит ли это слово некоторому фиксированному множеству.

Замечание 2.1. Оракул и подсказка — это не одно и то же: на разных словах одинаковой длины "ответы" оракула могут быть разными, в то время как подсказка зависит лишь от длины слова. И только унарный алфавит стирает разницу между оракулом и подсказкой.

3 Машины Тьюринга

Вспомним нашего старого друга из предыдущего семестра:

Определение 3.1. *Машина Тьюринга (MT) — это набор $(A, Q, Q_f, \delta, q_0, \Lambda)$, где*

- A — конечное множество, называемое алфавитом MT
- Q — конечное множество, называемое множеством состояний MT: $A \cap Q = \emptyset$
- Q_f — множество финальных состояний MT: $Q_f \subseteq Q$
- δ — функция (таблица) переходов: $\delta : A \times Q \rightarrow A \times \{-1, 0, 1\} \times Q$
- q_0 — начальное состояние: $q_0 \in Q$
- Λ — пустой символ: $\Lambda \in A$

Машины Тьюринга бывают разные:

- *Детерминированные (ДМТ) и недетерминированные (НМТ).*
В НМТ, в отличие от ДМТ, переходы не опеределены, то есть, паре "символ + состояние" может соответствовать несколько правил перехода (или не одного).

$$\text{ДМТ: } A^k \times Q \rightarrow A^k \times \{1, 0, -1\}^k \times Q$$

$$\text{НМТ: } A^k \times Q \rightarrow 2^{A^k \times \{1, 0, -1\}^k \times Q}$$

здесь k — количество лент в МТ.

- *Одноленточные и k -ленточные МТ.*
- *МТ с лентами, бесконечными в одну или в обе стороны.*
- *МТ с разными длинами алфавита* (то есть в алфавит МТ могут быть добавлены дополнительные служебные символы).
- *Бывает удобно различать ленты по назначению: входная, рабочее, выходная. И алфавиты — соответственно: входной, рабочий и выходной.*

Замечание 3.1. *Мы будем использовать k -ленточные детерминированные и недетерминированные машины Тьюринга. Количество лент выбираем из удобства, но для каждого алгоритма оно фиксировано и не зависит от входа.*

4 Асимптотические обозначения

Вспомним алгосы первого семестра, друзья мои: пусть функции f и g таковы, что $f, g : \mathbb{N} \rightarrow \mathbb{N}$, тогда:

- $f(n) = O(g(n))$, если $\exists C > 0, \exists N \in \mathbb{N} : \forall n > N \hookrightarrow f(n) < C \cdot g(n)$
- $f(n) = \Omega(g(n))$, если $\exists c > 0, \exists N \in \mathbb{N} : \forall n > N \hookrightarrow f(n) > c \cdot g(n)$
- $f(n) = \Theta(g(n))$, если $\exists c > 0, \exists C > 0, \exists N \in \mathbb{N} : \forall n > N \hookrightarrow c \cdot g(n) < f(n) < C \cdot g(n)$
- $f(n) = o(g(n))$, если $\forall c > 0 \exists N \in \mathbb{N} : \forall n > N \hookrightarrow f(n) < c \cdot g(n)$
- $f(n) = \omega(g(n))$, если $\forall C > 0 : \exists N \in \mathbb{N} : \forall n > N \hookrightarrow f(n) > C \cdot g(n)$

Помимо базовых, используются следующие синтетические обозначения:

- $f(n) = \text{poly}(g(n))$, если для некоторой константы c выполнено $f(n) = O((g(n))^c)$
- $f(n) = \text{polylog}(g(n))$, если для некоторой константы c выполнено $f(n) = O((\log g(n))^c)$
- $f(n) = \tilde{O}(g(n))$, если $f(n) = O(g(n) \cdot \text{polylog}(g(n)))$

5 Распознавание языков

Определение 5.1. Задача распознавания языка: По слову x нужно выяснить, принадлежит ли оно языку $L \subset \Sigma^*$. Алгоритм должен возвращать 1, если $x \in L$, и 0, если $x \notin L$.

Определение 5.2. Машина распознаёт язык A за время $T(n)$, если она принимает все слова, лежащие в A , отвергает все слова, не лежащие в A , и на каждом слове x работает не больше $T(|x|)$ шагов.

Замечание 5.1. Здесь измеряется сложность в *худшем случае*: нужно, чтобы максимальное время работы на словах длины n не превысило $T(n)$.